CSC 384/CSC 584

Database Design - Winter 2026

Library Management System (LMS)
Final Project Report

Project Title:

Library Management System (LMS)

Submitted by: Shayan Shoukat & Chaima Zaghouani

Table of Contents

PhaseI

1. Project Title and Description
2. Application Requirements
3. ER Schema Design
    3.1 Entity Definitions
    3.2 Relationship Definitions
    3.3 Complete ER Diagram
4. Group Member Contributions

PhaseII

1. SQL DDL Statements
2. Sample Data
3. Data in Tables
    3.1 Table: Staff
    3.2 Table: Readers
    3.3 Table: Series
    3.4 Table: Books
    3.5 Table: Book_Copies
    3.6 Table: Transactions
    3.7 Table: Fines
    3.8 Table: Reservations
4. Group Member Contributions

PhaseIII

1. Introduction
2. Overview of Operations
3. Scenarios and SQL Statements
4. Application UI Demonstration
5. Group Member Contributions

Conclusion

Phase I

1. Project Title and Description

Project Title

Library Management System (LMS)

Project Description

The Library Management System is a comprehensive database-driven application designed to manage the day-to-day operations of a modern library. The system maintains detailed records of books, library members, staff, and series while facilitating core library functions such as member registration, book borrowing, reservations, and fine management.

The system distinguishes between bibliographic information (the conceptual book with its title, ISBN, author, etc.) and physical book copies (the actual items on library shelves). This design reflects real-world library operations where multiple physical copies of the same book title exist in the collection. Each physical copy can have its own status (Available, Borrowed, Reserved, or Damaged) and shelf location, enabling accurate inventory tracking.

The LMS serves two primary user types: library staff and library readers (members). Staff members authenticate through secure login credentials to access administrative functions including inventory management, borrowing/return processing, member account oversight, and fine administration. Readers register with their personal information (name, email, phone number, and address) to access library services such as browsing available books, borrowing materials, placing reservations on unavailable titles, and managing their accounts including fine payments.

The borrowing process is comprehensively tracked through transaction records that link readers to specific book copies, recording crucial details such as borrow date, due date, return date, and transaction status. When desired books are unavailable, readers can place reservations that queue them for access when copies become available. The system automatically generates fines for overdue materials, tracking fine amounts, payment status, and payment dates.

This project demonstrates how a properly designed relational database can streamline library operations, reduce manual workload, and provide reliable access to information for both staff and patrons.

2. Application Requirements

Functional Requirements

Book and Inventory Management:

- The system must store bibliographic information for books including ISBN, title, author, edition, category, and language.
- The system must track multiple physical copies of the same book title, with each copy having its own status and shelf location.
- The system must maintain accurate real-time inventory of book availability (Available, Borrowed, Reserved, or Damaged).
- Staff must be able to add, update, and remove books and book copies from the catalog.

Member Management:

- The system must allow new readers to register with personal information (name, email, phone, address).
- The system must track membership date and account status for each reader.
- Staff must be able to view and update reader account information.
- The system must support a maximum of 10,000 active reader accounts.

Borrowing Operations:

- Staff must be able to check out books to readers, creating borrowing transaction records.
- The system must record borrow date, due date (typically 14 days from borrow date), and return date.
- Readers may borrow a maximum of 5 books simultaneously.
- Staff must be able to process returns and update book copy status accordingly.
- The system must track which staff member processed each borrowing/return transaction.

Reservation System:

- Readers must be able to reserve books that are currently unavailable.
- The system must queue reservations and notify when books become available.
- Reservations must expire after 7 days if not fulfilled.
- The system must track reservation date and status for each reservation.

Fine Management:

- The system must automatically calculate fines for overdue books per day.
- The system must track fine amount, fine date, payment status, and payment date.
- Staff must be able to record fine payments and update payment status.
- The system must display outstanding fine balance for each reader account.

Staff and Authentication:

- Staff members must authenticate using unique login credentials.

- The system must track staff roles (e.g., Librarian, Assistant, Administrator).
- The system must log last login date for security auditing.

Series Information:

- The system must maintain series records including series name and publisher.
- Each book must be associated with exactly one series.

3. ER Schema Design

3.1 Entity Definitions

The Library Management System consists of seven primary entities:

Entity 1: BOOKS

Description: Represents bibliographic information about book titles in the library catalog.

Attribute	Description
Book_ID	Primary Key. Unique identifier for each book title.
ISBN	International Standard Book Number.
Title	Book title.
Author	Author name(s).
Edition	Edition number or description.
Category	Genre or subject category (e.g., Fiction, Science, History).
Language	Language of publication.
Publication_Year	Year the book was published.

Entity 2: BOOK_COPIES

Description: Represents individual physical copies of books in the library's inventory. Each book title can have multiple copies.

Attribute	Description
Copy_ID	Primary Key. Unique identifier for each physical book copy.
Status	Current status: Available, Borrowed, Reserved, or Damaged.
Acquisition_Date	Date the copy was added to library inventory.
Shelf_Location	Physical location in the library (e.g., Aisle 3, Shelf B).

Entity 3: READERS

Description: Represents library members who can borrow books and use library services.

Attribute	Description
Member_ID	Primary Key. Unique identifier for each library member.
Name	Full name of the reader.
Email	Email address for communication.
Phone_Number	Contact phone number.
Address	Residential address.
Membership_Date	Date when the member joined the library.
Account_Status	Status: Active or Suspended.
Current_Fine_Balance	Total outstanding fines owed by the member.

Entity 4: STAFF

Description: Represents library staff members who manage library operations and serve readers.

Attribute	Description
Staff_ID	Primary Key. Unique identifier for each staff member.
Name	Full name of the staff member.
Email	Official email address.
Login_ID	Unique username for system authentication.
Password	Encrypted password for secure login.
Role	Job role (e.g., Librarian, Assistant, Administrator).
Last_Login_Date	Timestamp of most recent login.
Hire_Date	Date when the staff member was hired.
Department	Department within the library (e.g., Circulation, Reference).

Entity 5: SERIES

Description: Represents a collection or sequence of books that are published under the same series name.

Attribute	Description
Series_ID	Primary Key. Unique identifier for each series.
SeriesName	Name of the book series (e.g., *Harry Potter*, *Lord of the Rings*).
Publisher	Name of the publishing company that publishes this series.

Entity 6: TRANSACTION

Description: Represents borrowing transactions. This entity replaces the M:N BORROW relationship from the original design, providing better tracking and historical records.

Attribute	Description
Transaction_ID	Primary Key. Unique identifier for each transaction.
Borrow_Date	Date when the book was checked out.
Due_Date	Expected return date (typically 14 days from borrow date).
Return_Date	Actual date the book was returned (NULL if still borrowed).
Status	Transaction status: Active or Returned.

Entity 7: FINES

Description: Represents fines assessed to readers. This is now a proper entity linked to transactions, rather than an M:N relationship.

Attribute	Description
Fine_ID	Primary Key. Unique identifier for each fine.
Fine_Amount	Monetary amount of the fine.
Fine_Date	Date when the fine was assessed.
Payment_Status	Status: Paid or Unpaid.
Payment_Date	Date when the fine was paid (NULL if unpaid).

3.2 Relationship Definitions

The revised Library Management System includes the following relationships:

Relationship 1: BORROWS

Type: One-to-Many (1:M) - READERS to TRANSACTION

Description: A reader can have multiple borrowing transactions over time, but each transaction belongs to exactly one reader.

Relationship 2: IS_BORROWED

Type: One-to-Many (1:M) - BOOK_COPIES to TRANSACTION

Description: A book copy can be involved in multiple transactions over its lifetime, but each transaction involves exactly one book copy.

Relationship 3: PROCESSES

Type: One-to-Many (1:M) - STAFF to TRANSACTION

Description: A staff member can process multiple transactions, but each transaction is processed by exactly one staff member.

Relationship 4: HAS_FINE

Type: One-to-One (1:1) - TRANSACTION to FINES

Description: A transaction may have zero or one fine (if the book was returned late), and each fine is associated with exactly one transaction.

Relationship 5: RESERVE

Type: Many-to-Many (M:N) - READERS to BOOKS

Description: Represents reservations placed by readers on books that are currently unavailable. A reader can reserve multiple books, and a book can have multiple reservations queued.

Attributes:

- Reservation_ID
- Reservation_Date: Date when the reservation was placed
- Expiry_Date: Date when the reservation expires
- Status: Active, Fulfilled, Cancelled, or Expired

Relationship 6: HAS_COPIES

Type: One-to-Many (1:M) - BOOKS to BOOK_COPIES

Description: A book title can have multiple physical copies in the library, but each physical copy belongs to exactly one book title.

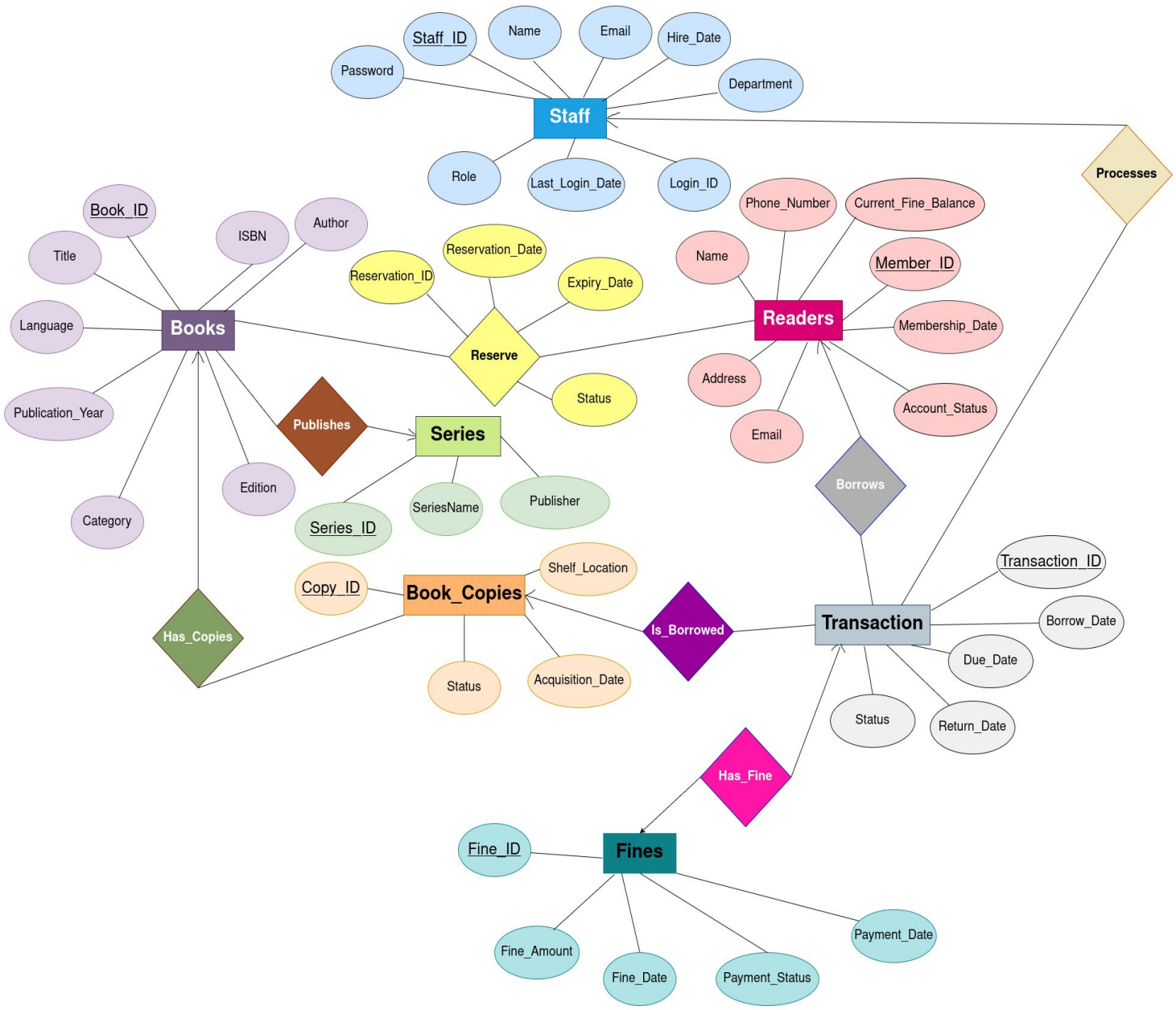
Relationship 7: PUBLISHES

Type: One-to-Many (1:M) – Series to BOOKS

Description: A series can include many books, but each book belongs to exactly one series.

3.3 Complete ER Diagram

The complete ER diagram illustrates all entities, attributes, and relationships in the Library Management System.



4. Group Member Contributions

This section documents the specific contributions of each group member to Phase I of the project:

Member 1: Shayan Shoukat

- Authored the project description and application requirements

Member 2: Chaima Zaghouani

- Prepared the initial ER diagram

Collaborative Revision:

- Both team members reviewed and contributed to the design improvements outlined in Section 3.

Phase II

1. SQL DDL Statements

Starting from our ER schema designed in Phase I, which contains 7 entity sets and 7 relationships, one might initially expect a total of 14 tables. However, the number of tables we actually create depends on the cardinality of each relationship, not just the count of relationships.

Out of our 7 relationships, 6 of them are either One-to-Many (1:M) or One-to-One (1:1). For these types of relationships, no new table is needed. Instead, the foreign key is simply added to the table on the "many" side, linking it back to the table on the "one" side. The 7 entity tables combined with these 6 absorbed relationships still give us 7 tables. The breakdown is as follows:

Relationship	Type	Translation Decision
BORROWS (Readers → Transactions)	1:M	Foreign key Member_ID added to Transactions table
IS_BORROWED (Book_Copies → Transactions)	1:M	Foreign key Copy_ID added to Transactions table
PROCESSES (Staff → Transactions)	1:M	Foreign key Staff_ID added to Transactions table
HAS_FINE (Transactions → Fines)	1:1	Foreign key Transaction_ID (UNIQUE) added to Fines table
HAS_COPIES (Books → Book_Copies)	1:M	Foreign key Book_ID added to Book_Copies table
PUBLISHES (Series → Books)	1:M	Foreign key Series_ID added to Books table
RESERVE (Readers ↔ Books)	M:N	New table created: Reservations — this is our 8th table

The only relationship that requires a new table is RESERVE, because it is a Many-to-Many (M:N) relationship between Readers and Books. A reader can reserve multiple books, and a book can have multiple reservations queued. This cannot be represented by simply adding a foreign key to one side, so we create a dedicated Reservations table that holds the foreign keys of both sides along with the reservation attributes (Reservation_ID, Reservation_Date, Expiry_Date, Status).

Therefore, the final count is: 7 entity tables + 1 new table for the M:N RESERVE relationship = 8 tables total.

The following Oracle SQL DDL statements implement all 8 tables. The script was tested on Oracle SQL Developer and ran successfully as expected:

```
-- Drop tables with CASCADE CONSTRAINTS to handle foreign key dependencies
DROP TABLE Reservations   CASCADE CONSTRAINTS;
DROP TABLE Fines          CASCADE CONSTRAINTS;
DROP TABLE Transactions   CASCADE CONSTRAINTS;
DROP TABLE Book_Copies    CASCADE CONSTRAINTS;
DROP TABLE Books          CASCADE CONSTRAINTS;
DROP TABLE Series         CASCADE CONSTRAINTS;
DROP TABLE Readers        CASCADE CONSTRAINTS;
DROP TABLE Staff          CASCADE CONSTRAINTS;

-- ============================================================
-- Table 1: STAFF
-- ============================================================
CREATE TABLE Staff (
   Staff_ID        INT             PRIMARY KEY,
   Name            VARCHAR2(100),
   Email           VARCHAR2(100),
   Login_ID        VARCHAR2(50)    UNIQUE,
   Password        VARCHAR2(255),
   Role            VARCHAR2(50),
   Last_Login_Date DATE,
   Hire_Date       DATE,
   Department      VARCHAR2(100)
);
-- ============================================================
-- Table 2: READERS
-- ============================================================
CREATE TABLE Readers (
   Member_ID            INT             PRIMARY KEY,
   Name                 VARCHAR2(100),
   Email                VARCHAR2(100),
   Phone_Number         VARCHAR2(20),
   Address              VARCHAR2(255),
   Membership_Date      DATE,
   Account_Status       VARCHAR2(20),
   Current_Fine_Balance REAL            DEFAULT 0
);
-- ============================================================
```

```
-- ============================================================
-- Table 3: SERIES
-- ============================================================
CREATE TABLE Series (
   Series_ID   INT PRIMARY KEY,
   SeriesName  VARCHAR2(100),
   Publisher   VARCHAR2(100)
);
-- ============================================================
-- Table 4: BOOKS
-- ============================================================
CREATE TABLE Books (
   Book_ID INT PRIMARY KEY,
   ISBN VARCHAR2(20) UNIQUE,
   Title VARCHAR2(255),
   Author VARCHAR2(100),
   Edition VARCHAR2(50),
   Category VARCHAR2(100),
   Language VARCHAR2(50),
   Publication_Year INT,
   Series_ID INT REFERENCES Series(Series_ID)
);
-- ============================================================
-- Table 5: BOOK_COPIES
-- ============================================================
CREATE TABLE Book_Copies (
   Copy_ID         INT            PRIMARY KEY,
   Status          VARCHAR2(20),
   Acquisition_Date DATE,
   Shelf_Location   VARCHAR2(100),
   Book_ID         INT            REFERENCES Books(Book_ID)
);
-- ============================================================
-- Table 6: TRANSACTIONS
-- ============================================================
CREATE TABLE Transactions (
   Transaction_ID INT         PRIMARY KEY,
   Borrow_Date    DATE,
   Due_Date       DATE,
   Return_Date    DATE,
   Status         VARCHAR2(20),
   Member_ID      INT         REFERENCES Readers(Member_ID),
   Copy_ID        INT         REFERENCES Book_Copies(Copy_ID),
   Staff_ID       INT         REFERENCES Staff(Staff_ID)
);
```

```
-- ============================================================
```

```
-- Table 7: FINES
-- (Transaction_ID is UNIQUE to enforce the 1:1 HAS_FINE relationship)
-- ============================================================
CREATE TABLE Fines (
    Fine_ID        INT          PRIMARY KEY,
    Fine_Amount    REAL,
    Fine_Date      DATE,
    Payment_Status VARCHAR2(20),
    Payment_Date   DATE,
    Transaction_ID INT          UNIQUE REFERENCES Transactions(Transaction_ID)
);
-- ============================================================
-- Table 8: RESERVATIONS
-- (Resolves the M:N RESERVE relationship between Readers and Books)
-- ============================================================
CREATE TABLE Reservations (
    Reservation_ID   INT          PRIMARY KEY,
    Reservation_Date DATE,
    Expiry_Date      DATE,
    Status           VARCHAR2(20),
    Member_ID        INT          REFERENCES Readers(Member_ID),
    Book_ID          INT          REFERENCES Books(Book_ID)
);
```

2. Sample Data

To populate our tables with test data, there are generally two approaches available in Oracle SQL:

Approach 1: INSERT INTO Statements

The first approach is to write individual **INSERT INTO** statements for each row, specifying the values directly in SQL. This is the approach we used in our homework assignments and is the standard method for inserting rows one at a time.

Approach 2: Loading Data from a CSV File

The second approach is to **load data from an external CSV file** using Oracle's SQL*Loader tool or the SQL Developer import wizard, similar to what we did in Homework 1. This method is more efficient when dealing with large datasets that already exist in spreadsheet format.

For this project, we opted for the INSERT INTO approach for two reasons. First, we wanted to practice and reinforce our familiarity with the Oracle DML syntax, as writing the statements manually deepens our understanding of how data maps to each table's column structure.

Second, we could not find an existing dataset that matched exactly the structure and constraints of our LMS database. For example, the specific status values (Active, Borrowed, Reserved, Damaged), the relationships between copies and transactions, and the fine calculation logic all required custom, hand-crafted data to be consistent and meaningful.

The following INSERT statements populate all 8 tables with realistic sample data covering 5 series, 5 staff members, 8 readers, 10 books, 21 book copies, 10 transactions, 2 fines, and 6 reservations. The script was tested on Oracle SQL Developer and ran successfully as expected:

```
-- Disables the '&' substitution variable prompt in Oracle SQL Developer
SET DEFINE OFF;
-- Column order follows DDL exactly:
--   Staff:       (Staff_ID, Name, Email, Login_ID, Password, Role, Last_Login_Date, Hire_Date, Department)
--   Readers:     (Member_ID, Name, Email, Phone_Number, Address, Membership_Date, Account_Status, Current_Fine_Balance)
-- Series: (Series_ID, SeriesName, Publisher)
-- Books: (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
--   Book_Copies:  (Copy_ID, Status, Acquisition_Date, Shelf_Location, Book_ID)
--   Transactions: (Transaction_ID, Borrow_Date, Due_Date, Return_Date, Status, Member_ID, Copy_ID, Staff_ID)
--   Fines:       (Fine_ID, Fine_Amount, Fine_Date, Payment_Status, Payment_Date, Transaction_ID)
--   Reservations: (Reservation_ID, Reservation_Date, Expiry_Date, Status, Member_ID, Book_ID)
-- ============================================================
-- ============================================================
-- SERIES (5 real-world book series)
-- ============================================================
INSERT INTO Series VALUES (1, 'Penguin Classics',       'Penguin Random House');
INSERT INTO Series VALUES (2, 'Harper Perennial',       'HarperCollins');
INSERT INTO Series VALUES (3, 'Scribner',               'Simon  Schuster');
INSERT INTO Series VALUES (4, 'Learning Series',        'O''Reilly Media');
INSERT INTO Series VALUES (5, 'Computer Science Texts',  'Springer');
-- ============================================================
```

```
-- STAFF (5 employees)
-- Roles per report: Librarian | Assistant | Administrator
-- Departments: Circulation | Reference | Management
-- Last_Login_Date: recent realistic dates
-- ============================================================

INSERT INTO Staff VALUES (101, 'Margaret Thompson', 'mthompson@library.org', 'mthompson', 'hashed_mthompson_pw',
'Administrator', TO_DATE('2026-03-05', 'YYYY-MM-DD'), TO_DATE('2018-06-01', 'YYYY-MM-DD'), 'Management');
INSERT INTO Staff VALUES (102, 'David Okafor',      'dokafor@library.org',   'dokafor',   'hashed_dokafor_pw',   'Librarian',
TO_DATE('2026-03-04', 'YYYY-MM-DD'), TO_DATE('2020-09-15', 'YYYY-MM-DD'), 'Circulation');
INSERT INTO Staff VALUES (103, 'Priya Nair',        'pnair@library.org',     'pnair',     'hashed_pnair_pw',     'Librarian',
TO_DATE('2026-03-05', 'YYYY-MM-DD'), TO_DATE('2021-01-10', 'YYYY-MM-DD'), 'Reference');
INSERT INTO Staff VALUES (104, 'Carlos Rivera',     'crivera@library.org',   'crivera',   'hashed_crivera_pw',   'Assistant',
TO_DATE('2026-03-03', 'YYYY-MM-DD'), TO_DATE('2022-08-22', 'YYYY-MM-DD'), 'Circulation');
INSERT INTO Staff VALUES (105, 'Aisha Mensah',      'amensah@library.org',   'amensah',   'hashed_amensah_pw',   'Assistant',
TO_DATE('2026-03-05', 'YYYY-MM-DD'), TO_DATE('2023-03-05', 'YYYY-MM-DD'), 'Circulation');
```

```
-- ============================================================
-- READERS (8 members)
-- Account_Status per report: Active | Suspended
-- Current_Fine_Balance: 0 if no unpaid fines; matches Fines table
--   Reader 207 (Lucas Fernandez) has unpaid fine of $5.00  -> balance = 5.00
--   Reader 205 (Daniel Kowalski) fine is Paid              -> balance = 0
-- ============================================================
INSERT INTO Readers VALUES (201, 'James O''Brien',   'jobrien@gmail.com',        '(313) 555-0142', '412 Woodward Ave, Detroit, MI',    TO_DATE('2023-09-01', 'YYYY-MM-DD'), 'Active',    0);
INSERT INTO Readers VALUES (202, 'Fatima Al-Hassan', 'fatima.alhassan@yahoo.com', '(810) 555-0188', '78 Court St, Flint, MI', TO_DATE('2023-11-15', 'YYYY-MM-DD'), 'Active',    0);
INSERT INTO Readers VALUES (203, 'Michael Chen',     'mchen@outlook.com',         '(734) 555-0211', '930 State St, Ann Arbor, MI',      TO_DATE('2024-01-20', 'YYYY-MM-DD'), 'Active',    0);
INSERT INTO Readers VALUES (204, 'Sophia Tremblay',  'stremblay@gmail.com',       '(248) 555-0307', '15 Maple Blvd, Southfield, MI',    TO_DATE('2024-03-08', 'YYYY-MM-DD'), 'Active',    0);
INSERT INTO Readers VALUES (205, 'Daniel Kowalski',  'dkowalski@gmail.com',       '(586) 555-0419', '202 Gratiot Ave, Warren, MI',      TO_DATE('2024-05-17', 'YYYY-MM-DD'), 'Active',    0);
INSERT INTO Readers VALUES (206, 'Amara Diallo',     'amaradiallo@gmail.com',     '(313) 555-0533', '55 Grand River Ave, Detroit, MI',  TO_DATE('2024-07-30', 'YYYY-MM-DD'), 'Active',    0);
INSERT INTO Readers VALUES (207, 'Lucas Fernandez',  'lfernandez@hotmail.com',    '(810) 555-0627', '19 Saginaw St, Flint, MI', TO_DATE('2024-09-12', 'YYYY-MM-DD'), 'Suspended', 5.00);
INSERT INTO Readers VALUES (208, 'Emily Nakamura',   'enakamura@gmail.com',       '(734) 555-0748', '301 Packard St, Ann Arbor, MI',    TO_DATE('2025-01-03', 'YYYY-MM-DD'), 'Active',    0);
```

```
-- ============================================================
-- BOOKS (10 real books with correct ISBNs)
-- Category examples per report: Fiction | History | Biography | Technology

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (301, '9780141182636', '1984', 'George Orwell', '1st', 'Fiction', 'English', 1949, 1);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (302, '9780062316097', 'Sapiens: A Brief History of Humankind', 'Yuval Noah Harari', '1st', 'History', 'English', 2015, 2);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (303, '9780743273565', 'The Great Gatsby', 'F. Scott Fitzgerald', '1st', 'Fiction', 'English', 1925, 3);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (304, '9780743454537', 'Steve Jobs', 'Walter Isaacson', '1st', 'Biography', 'English', 2011, 3);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (305, '9781492051367', 'Learning SQL', 'Alan Beaulieu', '3rd', 'Technology', 'English', 2020, 4);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (306, '9780136019701', 'Artificial Intelligence: A Modern Approach', 'Russell  Norvig', '4th', 'Technology', 'English', 2020, 5);
```

```
INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (307, '9780385333481', 'The Handmaid''s Tale', 'Margaret Atwood', '1st', 'Fiction', 'English', 1985, 1);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (308, '9780525559474', 'The Midnight Library', 'Matt Haig', '1st', 'Fiction', 'English', 2020, 2);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (309, '9780385737951', 'The Hunger Games', 'Suzanne Collins', '1st', 'Fiction', 'English', 2008, 1);

INSERT INTO Books (Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year, Series_ID)
VALUES (310, '9780134685991', 'Effective Java', 'Joshua Bloch', '3rd', 'Technology', 'English', 2018, 4);
```

```
-- ============================================================
-- BOOK_COPIES (21 copies, 2-3 per book)
-- Status per report: Available | Borrowed | Reserved | Damaged
-- Borrowed copies are linked to Active transactions below
-- ============================================================

-- 1984 (3 copies)
INSERT INTO Book_Copies VALUES (5001, 'Available', TO_DATE('2022-08-01', 'YYYY-MM-DD'), 'Aisle 1 / Shelf A', 301);
INSERT INTO Book_Copies VALUES (5002, 'Borrowed',  TO_DATE('2022-08-01', 'YYYY-MM-DD'), 'Aisle 1 / Shelf A', 301);
INSERT INTO Book_Copies VALUES (5003, 'Damaged',   TO_DATE('2022-08-01', 'YYYY-MM-DD'), 'Aisle 1 / Shelf A', 301);
-- Sapiens (2 copies)
INSERT INTO Book_Copies VALUES (5004, 'Available', TO_DATE('2023-01-15', 'YYYY-MM-DD'), 'Aisle 1 / Shelf B', 302);
INSERT INTO Book_Copies VALUES (5005, 'Borrowed',  TO_DATE('2023-01-15', 'YYYY-MM-DD'), 'Aisle 1 / Shelf B', 302);
-- The Great Gatsby (2 copies)
INSERT INTO Book_Copies VALUES (5006, 'Available', TO_DATE('2021-05-20', 'YYYY-MM-DD'), 'Aisle 2 / Shelf A', 303);
INSERT INTO Book_Copies VALUES (5007, 'Available', TO_DATE('2021-05-20', 'YYYY-MM-DD'), 'Aisle 2 / Shelf A', 303);
-- Steve Jobs (2 copies)
INSERT INTO Book_Copies VALUES (5008, 'Borrowed',  TO_DATE('2023-03-10', 'YYYY-MM-DD'), 'Aisle 2 / Shelf B', 304);
INSERT INTO Book_Copies VALUES (5009, 'Available', TO_DATE('2023-03-10', 'YYYY-MM-DD'), 'Aisle 2 / Shelf B', 304);
-- Learning SQL (2 copies)
INSERT INTO Book_Copies VALUES (5010, 'Available', TO_DATE('2024-01-08', 'YYYY-MM-DD'), 'Aisle 3 / Shelf A', 305);
INSERT INTO Book_Copies VALUES (5011, 'Borrowed',  TO_DATE('2024-01-08', 'YYYY-MM-DD'), 'Aisle 3 / Shelf A', 305);
-- AI: A Modern Approach (2 copies)
INSERT INTO Book_Copies VALUES (5012, 'Available', TO_DATE('2024-02-14', 'YYYY-MM-DD'), 'Aisle 3 / Shelf B', 306);
INSERT INTO Book_Copies VALUES (5013, 'Available', TO_DATE('2024-02-14', 'YYYY-MM-DD'), 'Aisle 3 / Shelf B', 306);
-- The Handmaid's Tale (2 copies)
INSERT INTO Book_Copies VALUES (5014, 'Available', TO_DATE('2022-11-01', 'YYYY-MM-DD'), 'Aisle 4 / Shelf A', 307);
INSERT INTO Book_Copies VALUES (5015, 'Borrowed',  TO_DATE('2022-11-01', 'YYYY-MM-DD'), 'Aisle 4 / Shelf A', 307);
-- The Midnight Library (2 copies)
INSERT INTO Book_Copies VALUES (5016, 'Available', TO_DATE('2024-06-20', 'YYYY-MM-DD'), 'Aisle 4 / Shelf B', 308);
INSERT INTO Book_Copies VALUES (5017, 'Available', TO_DATE('2024-06-20', 'YYYY-MM-DD'), 'Aisle 4 / Shelf B', 308);
-- The Hunger Games (2 copies)
INSERT INTO Book_Copies VALUES (5018, 'Borrowed',  TO_DATE('2023-07-05', 'YYYY-MM-DD'), 'Aisle 5 / Shelf A', 309);
```

```
INSERT INTO Book_Copies VALUES (5019, 'Available', TO_DATE('2023-07-05', 'YYYY-MM-DD'), 'Aisle 5 / Shelf A', 309);
-- Effective Java (2 copies)
INSERT INTO Book_Copies VALUES (5020, 'Available', TO_DATE('2024-09-01', 'YYYY-MM-DD'), 'Aisle 5 / Shelf B', 310);
INSERT INTO Book_Copies VALUES (5021, 'Borrowed',  TO_DATE('2024-09-01', 'YYYY-MM-DD'), 'Aisle 5 / Shelf B', 310);
-- ============================================================
```

-- TRANSACTIONS (10 records)

```
-- Status per report: Active | Returned
-- Due_Date = Borrow_Date + 14 days (as per requirements)
-- Active: Return_Date = NULL
-- ============================================================

-- Active (5 ongoing borrows - Return_Date is NULL)
INSERT INTO Transactions VALUES (4001, TO_DATE('2026-02-10', 'YYYY-MM-DD'), TO_DATE('2026-02-24', 'YYYY-MM-DD'),
NULL,                           'Active',   201, 5002, 102);
INSERT INTO Transactions VALUES (4002, TO_DATE('2026-02-15', 'YYYY-MM-DD'), TO_DATE('2026-03-01', 'YYYY-MM-DD'),
NULL,                           'Active',   203, 5005, 103);
INSERT INTO Transactions VALUES (4003, TO_DATE('2026-02-20', 'YYYY-MM-DD'), TO_DATE('2026-03-06', 'YYYY-MM-DD'),
NULL,                           'Active',   206, 5008, 104);
INSERT INTO Transactions VALUES (4004, TO_DATE('2026-02-25', 'YYYY-MM-DD'), TO_DATE('2026-03-11', 'YYYY-MM-DD'),
NULL,                           'Active',   208, 5011, 102);
INSERT INTO Transactions VALUES (4005, TO_DATE('2026-02-28', 'YYYY-MM-DD'), TO_DATE('2026-03-14', 'YYYY-MM-DD'),
NULL,                           'Active',   204, 5015, 105);

-- Returned on time (3 records - Return_Date before Due_Date)
INSERT INTO Transactions VALUES (4006, TO_DATE('2026-01-05', 'YYYY-MM-DD'), TO_DATE('2026-01-19',  'YYYY-MM-DD'),
TO_DATE('2026-01-17', ' YYYY-MM-DD'), 'Returned', 201, 5010, 102);
INSERT INTO Transactions VALUES (4007, TO_DATE('2026-01-10',  'YYYY-MM-DD'), TO_DATE('2026-01-24',  'YYYY-MM-DD'),
TO_DATE('2026-01-22',  'YYYY-MM-DD'), 'Returned', 202, 5006, 103);
INSERT INTO Transactions VALUES (4008, TO_DATE('2026-01-20',  'YYYY-MM-DD'), TO_DATE('2026-02-03',  'YYYY-MM-DD'),
TO_DATE('2026-02-01',  'YYYY-MM-DD'), 'Returned', 204, 5016, 104);

-- Returned late (2 records - Return_Date after Due_Date -> triggers Fines)
-- 4009: 5 days late  (due Jan 15, returned Jan 20)
-- 4010: 10 days late (due Jan 26, returned Feb 05)
INSERT INTO Transactions VALUES (4009, TO_DATE('2026-01-01', 'YYYY-MM-DD'), TO_DATE('2026-01-15', 'YYYY-MM-DD'),
TO_DATE('2026-01-20', 'YYYY-MM-DD'), 'Returned', 205, 5018, 102);
INSERT INTO Transactions VALUES (4010, TO_DATE('2026-01-12', 'YYYY-MM-DD'), TO_DATE('2026-01-26', 'YYYY-MM-DD'),
TO_DATE('2026-02-05', 'YYYY-MM-DD'), 'Returned', 207, 5021, 105);
-- ============================================================
```

-- FINES (2 fines - only for late transactions 4009 and 4010)

```
-- Fine rate: $0.50/day overdue (per application requirements)
-- Payment_Status per report: Paid | Unpaid
-- Payment_Date = NULL if Unpaid
-- Reader 205 fine: Paid   -> Current_Fine_Balance = 0  (consistent)
-- Reader 207 fine: Unpaid -> Current_Fine_Balance = 5.00 (consistent)
-- ============================================================

INSERT INTO Fines VALUES (6001, 2.50, TO_DATE('2026-01-20', 'YYYY-MM-DD'), 'Paid',   TO_DATE('2026-01-22', 'YYYY-MM-DD'), 4009);
INSERT INTO Fines VALUES (6002, 5.00, TO_DATE('2026-02-05', 'YYYY-MM-DD'), 'Unpaid', NULL,                          4010);
-- ============================================================
```

-- RESERVATIONS (6 records)

```
-- Status per report: Active | Fulfilled | Cancelled | Expired
-- Expiry_Date = Reservation_Date + 7 days (per requirements)
-- Readers reserve books that are Borrowed or Damaged
-- ============================================================

INSERT INTO Reservations VALUES (7001, TO_DATE('2026-02-12', 'YYYY-MM-DD'), TO_DATE('2026-02-19', 'YYYY-MM-DD'), 'Active',    202, 301);
INSERT INTO Reservations VALUES (7002, TO_DATE('2026-02-14', 'YYYY-MM-DD'), TO_DATE('2026-02-21', 'YYYY-MM-DD'), 'Active',    206, 302);
INSERT INTO Reservations VALUES (7003, TO_DATE('2026-02-16', 'YYYY-MM-DD'), TO_DATE('2026-02-23', 'YYYY-MM-DD'), 'Active',    208, 304);
INSERT INTO Reservations VALUES (7004, TO_DATE('2026-01-10', 'YYYY-MM-DD'), TO_DATE('2026-01-17', 'YYYY-MM-DD'), 'Fulfilled', 201, 309);
INSERT INTO Reservations VALUES (7005, TO_DATE('2026-01-15', 'YYYY-MM-DD'), TO_DATE('2026-01-22', 'YYYY-MM-DD'), 'Expired',   203, 307);
INSERT INTO Reservations VALUES (7006, TO_DATE('2026-01-20', 'YYYY-MM-DD'), TO_DATE('2026-01-27', 'YYYY-MM-DD'), 'Cancelled', 207, 310);
```

3. Data in Tables

After successfully running both the DDL script and the INSERT script in Oracle SQL Developer, we verified the data by querying each table individually. The screenshots below show the contents of all 8 tables as displayed in Oracle SQL Developer, confirming that the data was inserted correctly and all foreign key relationships are intact.

3.1 Table: Staff

	STAFF_ID	NAME	EMAIL	LOGIN_ID	PASSWORD	ROLE	LAST_LOGIN_DATE	HIRE_DATE	DEPARTMENT
1	101	Margaret Thompson	mthompson@library.org	mthompson	hashed_mthompson_pw	Administrator	05-MAR-26	01-JUN-18	Management
2	102	David Okafor	dokafor@library.org	dokafor	hashed_dokafor_pw	Librarian	04-MAR-26	15-SEP-20	Circulation
3	103	Priya Nair	pnair@library.org	pnair	hashed_pnair_pw	Librarian	05-MAR-26	10-JAN-21	Reference
4	104	Carlos Rivera	crivera@library.org	crivera	hashed_crivera_pw	Assistant	03-MAR-26	22-AUG-22	Circulation
5	105	Aisha Mensah	amensah@library.org	amensah	hashed_amensah_pw	Assistant	05-MAR-26	05-MAR-23	Circulation

3.2 Table: Readers

	MEMBER_ID	NAME	EMAIL	PHONE_NUMBER	ADDRESS	MEMBERSHIP_DATE	ACCOUNT_STATUS	CURRENT_FINE_BALANCE
1	201	James O'Brien	jobrien@gmail.com	(313) 555-0142	412 Woodward Ave, Detroit, MI	01-SEP-23	Active	0
2	202	Fatima Al-Hassan	fatima.alhassan@yahoo.com	(810) 555-0188	78 Court St, Flint, MI	15-NOV-23	Active	0
3	203	Michael Chen	mchen@outlook.com	(734) 555-0211	930 State St, Ann Arbor, MI	20-JAN-24	Active	0
4	204	Sophia Tremblay	stremblay@gmail.com	(248) 555-0307	15 Maple Blvd, Southfield, MI	08-MAR-24	Active	0
5	205	Daniel Kowalski	dkowalski@gmail.com	(586) 555-0419	202 Gratiot Ave, Warren, MI	17-MAY-24	Active	0
6	206	Amara Diallo	amaradiallo@gmail.com	(313) 555-0533	55 Grand River Ave, Detroit, MI	30-JUL-24	Active	0
7	207	Lucas Fernandez	lfernandez@hotmail.com	(810) 555-0627	19 Saginaw St, Flint, MI	12-SEP-24	Suspended	5
8	208	Emily Nakamura	enakamura@gmail.com	(734) 555-0748	301 Packard St, Ann Arbor, MI	03-JAN-25	Active	0

3.3 Table: Series

	SERIES_ID	SERIESNAME	PUBLISHER
1	1	Penguin Classics	Penguin Random House
2	2	Harper Perennial	HarperCollins
3	3	Scribner	Simon Schuster
4	4	Learning Series	O'Reilly Media
5	5	Computer Science Texts	Springer

3.4 Table: Books

	BOOK_ID	ISBN	TITLE	AUTHOR	EDITION	CATEGORY	LANGUAGE	PUBLICATION_YEAR	SERIES_ID
1	301	9780141182636	1984	George Orwell	1st	Fiction	English	1949	1
2	302	9780062316097	Sapiens: A Brief History of Humankind	Yuval Noah Harari	1st	History	English	2015	2
3	303	9780743273565	The Great Gatsby	F. Scott Fitzgerald	1st	Fiction	English	1925	3
4	304	9780743454537	Steve Jobs	Walter Isaacson	1st	Biography	English	2011	3
5	305	9781492051367	Learning SQL	Alan Beaulieu	3rd	Technology	English	2020	4
6	306	9780136019701	Artificial Intelligence: A Modern Approach	Russell Norvig	4th	Technology	English	2020	5
7	307	9780385333481	The Handmaid's Tale	Margaret Atwood	1st	Fiction	English	1985	1
8	308	9780525559474	The Midnight Library	Matt Haig	1st	Fiction	English	2020	2
9	309	9780385737951	The Hunger Games	Suzanne Collins	1st	Fiction	English	2008	1
10	310	9780134685991	Effective Java	Joshua Bloch	3rd	Technology	English	2018	4

3.5 Table: Book_Copies

	COPY_ID	STATUS	ACQUISITION_DATE	SHELF_LOCATION	BOOK_ID
1	5001	Available	01-AUG-22	Aisle 1 / Shelf A	301
2	5002	Borrowed	01-AUG-22	Aisle 1 / Shelf A	301
3	5003	Damaged	01-AUG-22	Aisle 1 / Shelf A	301
4	5004	Available	15-JAN-23	Aisle 1 / Shelf B	302
5	5005	Borrowed	15-JAN-23	Aisle 1 / Shelf B	302
6	5006	Available	20-MAY-21	Aisle 2 / Shelf A	303
7	5007	Available	20-MAY-21	Aisle 2 / Shelf A	303
8	5008	Borrowed	10-MAR-23	Aisle 2 / Shelf B	304
9	5009	Available	10-MAR-23	Aisle 2 / Shelf B	304
10	5010	Available	08-JAN-24	Aisle 3 / Shelf A	305
11	5011	Borrowed	08-JAN-24	Aisle 3 / Shelf A	305
12	5012	Available	14-FEB-24	Aisle 3 / Shelf B	306
13	5013	Available	14-FEB-24	Aisle 3 / Shelf B	306
14	5014	Available	01-NOV-22	Aisle 4 / Shelf A	307
15	5015	Borrowed	01-NOV-22	Aisle 4 / Shelf A	307
16	5016	Available	20-JUN-24	Aisle 4 / Shelf B	308
17	5017	Available	20-JUN-24	Aisle 4 / Shelf B	308
18	5018	Borrowed	05-JUL-23	Aisle 5 / Shelf A	309
19	5019	Available	05-JUL-23	Aisle 5 / Shelf A	309
20	5020	Available	01-SEP-24	Aisle 5 / Shelf B	310
21	5021	Borrowed	01-SEP-24	Aisle 5 / Shelf B	310

3.6 Table: Transactions

	TRANSACTION_ID	MEMBER...	COPY_ID	STAFF...	BORROW_...	DUE_D...	RETURN...	STATUS
1	4001	201	5002	102	10-FEB-26	24-FEB-26	(null)	Active
2	4002	203	5005	103	15-FEB-26	01-MAR-26	(null)	Active
3	4003	206	5008	104	20-FEB-26	06-MAR-26	(null)	Active
4	4004	208	5011	102	25-FEB-26	11-MAR-26	(null)	Active
5	4005	204	5015	105	28-FEB-26	14-MAR-26	(null)	Active
6	4006	201	5010	102	05-JAN-26	19-JAN-26	17-JAN-26	Returned
7	4007	202	5006	103	10-JAN-26	24-JAN-26	22-JAN-26	Returned
8	4008	204	5016	104	20-JAN-26	03-FEB-26	01-FEB-26	Returned
9	4009	205	5018	102	01-JAN-26	15-JAN-26	20-JAN-26	Returned
10	4010	207	5021	105	12-JAN-26	26-JAN-26	05-FEB-26	Returned

3.7 Table: Fines

	FINE_ID	FINE_AMOUNT	FINE_DATE	PAYMENT_STATUS	PAYMENT_DATE	TRANSACTION_ID
1	6001	2.5	20-JAN-26	Paid	22-JAN-26	4009
2	6002	5	05-FEB-26	Unpaid	(null)	4010

3.8 Table: Reservations

	RESERVATION_ID	RESERVATION_DATE	EXPIRY_DATE	STATUS	MEMBER_ID	BOOK_ID
1	7001	12-FEB-26	19-FEB-26	Active	202	301
2	7002	14-FEB-26	21-FEB-26	Active	206	302
3	7003	16-FEB-26	23-FEB-26	Active	208	304
4	7004	10-JAN-26	17-JAN-26	Fulfilled	201	309
5	7005	15-JAN-26	22-JAN-26	Expired	203	307
6	7006	20-JAN-26	27-JAN-26	Cancelled	207	310

4. Group Member Contributions

For Phase II, both group members worked independently on the full set of deliverables before comparing and consolidating the final version. Specifically, each member individually wrote the SQL DDL CREATE TABLE statements and the INSERT INTO data statements from scratch, then we reviewed both versions side by side, discussed any differences in design decisions or data choices, and agreed on the final result presented in this report. This approach ensured that both members have a solid understanding of the relational schema and the Oracle SQL syntax used throughout Phase II.

Member	Contributions
Shayan Shoukat	Independently wrote the SQL DDL statements and INSERT data; contributed to the ER-to-Relational translation discussion; participated in the final review and consolidation
Chaima Zaghouani	Independently wrote the SQL DDL statements and INSERT data; collected Oracle screenshots; participated in the final review and consolidation; compiled the Phase II report
Collaborative	Both members compared their individual work, resolved differences, and jointly agreed on the final DDL, data, and report content presented here

Phase III

1. Introduction

In Phase III of this project, we defined and implemented a comprehensive set of operations for the Library Management System (LMS). These operations are designed to simulate realistic interactions between users and the database, supporting essential library functionalities such as reader management, book search, inventory tracking, borrowing and returning books, reservations, and fine monitoring.

The goal of this phase is to demonstrate how the database can be actively used through SQL Data Manipulation Language (DML) statements. The operations include a mix of retrieval (SELECT), insertion (INSERT), and update (UPDATE) queries, as well as more advanced queries involving aggregation and multiple related tables. This aligns with the project requirement to support meaningful and reasonably comprehensive operations.

All queries were written and tested using Oracle SQL. Oracle-specific features such as `SYSDATE` (to retrieve the current system date) and `TO_DATE` (for date conversion) were used where appropriate to ensure realistic and practical behavior.

2. Overview of Operations

The selected scenarios cover the core functionalities of the LMS, including:

- Reader management (registration and lookup)
- Book and inventory management
- Borrowing and return processing
- Reservation handling
- Monitoring overdue books and unpaid fines

These operations are based on the schema developed in Phases I and II, which includes the tables: **`Staff`, `Readers`, `Series`, `Books`, `Book_Copies`, `Transactions`, `Fines`, and `Reservations`.**

3. Scenarios and SQL Statements

Scenario 1: Retrieve Reader Information by Member ID

Description:
This operation retrieves the complete details of a library member based on their unique Member ID.

User Input: Member ID
Output: Reader details including name, contact information, account status, and fine balance

SQL Statement:

```
SELECT Member_ID, Name, Email, Phone_Number, Address, Membership_Date,
Account_Status, Current_Fine_Balance
FROM Readers
WHERE Member_ID = &member_id;
```

Scenario 2: Search Books by Title

Description:

This operation allows users to search for books using a keyword in the title.

SQL Statement:

```
SELECT Book_ID, ISBN, Title, Author, Edition, Category, Language, Publication_Year
FROM Books
WHERE LOWER(Title) LIKE LOWER('%' || '&title_keyword' || '%');
```

Scenario 3: Display All Copies of a Book

Description:

This operation retrieves all physical copies associated with a specific book.

SQL Statement:

```
SELECT Copy_ID, Status, Acquisition_Date, Shelf_Location
FROM Book_Copies
WHERE Book_ID = &book_id
ORDER BY Copy_ID;
```

Scenario 4: Register a New Reader

Description:
This operation inserts a new reader into the system.

SQL Statement:

```
INSERT INTO Readers (
Member_ID, Name, Email, Phone_Number, Address,
Membership_Date, Account_Status, Current_Fine_Balance
)
VALUES (
&member_id,
'&name',
'&email',
'&phone_number',
'&address',
TO_DATE('&membership_date', 'YYYY-MM-DD'),
'&account_status',
&fine_balance
 );
```

Scenario 5: Add a New Book

Description:
This operation inserts a new book record into the catalog.

SQL Statement:

```
INSERT INTO Books (
Book_ID, ISBN, Title, Author, Edition, Category,
Language, Publication_Year, Series_ID
)
VALUES (
&book_id,
'&isbn',
'&title',
'&author',
'&edition',
'&category',
'&language',
&publication_year,
&series_id
);
```

Scenario 6: Add a New Book Copy

Description:
This operation adds a new physical copy of a book.

SQL Statement:

```
INSERT INTO Book_Copies (
Copy_ID, Status, Acquisition_Date, Shelf_Location, Book_ID
)
VALUES (
&copy_id,
'&status',
TO_DATE('&acquisition_date', 'YYYY-MM-DD'),
'&shelf_location',
&book_id
);
```

Scenario 7: Borrow a Book

Description:
This operation records a borrowing transaction and updates the status of the selected book copy.

SQL Statements:

```
INSERT INTO Transactions (
Transaction_ID, Borrow_Date, Due_Date, Return_Date, Status,
Member_ID, Copy_ID, Staff_ID
)
VALUES (
&transaction_id,
SYSDATE,
SYSDATE + 14,
NULL,
'Active',
&member_id,
&copy_id,
&staff_id
);

UPDATE Book_Copies
SET Status = 'Borrowed'
WHERE Copy_ID = &copy_id;
```

Scenario 8: Return a Book

Description:

This operation updates the transaction and marks the book copy as available.

SQL Statements:

```
UPDATE Transactions
SET Return_Date = SYSDATE,
 Status = 'Returned'
WHERE Transaction_ID = &transaction_id;

 UPDATE Book_Copies
  SET Status = 'Available'
  WHERE Copy_ID = &copy_id;
```

Scenario 9: Place a Reservation

Description:

This operation allows a reader to reserve a book.

SQL Statement:

```
INSERT INTO Reservations (
Reservation_ID, Reservation_Date, Expiry_Date,
Status, Member_ID, Book_ID
)
VALUES (
&reservation_id,
SYSDATE,
SYSDATE + 7,
'Active',
&member_id,
&book_id
 );
```

Scenario 10: Cancel a Reservation

Description:
This operation updates the status of a reservation.

SQL Statement:

```
UPDATE Reservations
SET Status = 'Cancelled'
WHERE Reservation_ID = &reservation_id;
```

Scenario 11: Retrieve Overdue Active Transactions

Description:
This query identifies all currently overdue borrowed books.

SQL Statement:

```
SELECT t.Transaction_ID, r.Member_ID, r.Name, b.Title, t.Borrow_Date, t.Due_Date,
ROUND(SYSDATE - t.Due_Date) AS Days_Overdue
FROM Transactions t, Readers r, Book_Copies bc, Books b
WHERE t.Member_ID = r.Member_ID
AND t.Copy_ID = bc.Copy_ID
AND bc.Book_ID = b.Book_ID
AND t.Status = 'Active'
AND t.Due_Date < SYSDATE
ORDER BY Days_Overdue DESC;
```

Scenario 12: Retrieve Readers with Unpaid Fines

Description:
This query identifies readers with outstanding unpaid fines.

SQL Statement:

```
SELECT r.Member_ID,  r.Name, SUM(f.Fine_Amount) AS Total_Unpaid
 FROM Readers r, Transactions t, Fines f
 WHERE r.Member_ID = t.Member_ID
 AND t.Transaction_ID = f.Transaction_ID
 AND f.Payment_Status = 'Unpaid'
 GROUP BY r.Member_ID, r.Name
 ORDER BY Total_Unpaid DESC;
```

Scenario 13: Insert Fine Automatically

Description:
When a book is returned late → automatically create a fine.

SQL Statement:

```
INSERT INTO Fines (Fine_ID, Fine_Amount, Fine_Date, Payment_Status, Payment_Date,
Transaction_ID)
SELECT
&fine_id,
 ROUND((Return_Date - Due_Date) * 0.5, 2),
 SYSDATE,
 'Unpaid',
  NULL,
 Transaction_ID
 FROM Transactions
 WHERE (Return_Date > Due_Date) AND (Transaction_ID = &transaction_id);
```

Scenario 14: Update Reader Fine Balance Automatically

Description:
This operation recalculates and updates a reader's current outstanding fine balance based on all unpaid fines associated with that reader.

SQL Statement:

```
UPDATE Readers r
SET Current_Fine_Balance = (
SELECT NVL(SUM(f.Fine_Amount), 0)
FROM Transactions t, Fines f
WHERE t.Transaction_ID = f.Transaction_ID
AND t.Member_ID = r.Member_ID
AND f.Payment_Status = 'Unpaid'
)
WHERE r.Member_ID = &member_id;
```

Scenario 15: Prevent Borrowing if Reader is Suspended

Description:
This operation checks the account status of a reader before processing a borrowing request. Readers whose accounts are suspended are not allowed to borrow books.

SQL Statement:

```
SELECT Account_Status
FROM Readers
WHERE Member_ID = &member_id;
```

Scenario 16: Get Most Borrowed Books (Analytics)

Description:
This analytical query identifies the most frequently borrowed books in the library by counting the number of borrowing transactions associated with each title.

SQL Statement:

```
SELECT b.Title, COUNT(t.Transaction_ID) AS Borrow_Count
FROM Books b, Book_Copies bc, Transactions t
WHERE b.Book_ID = bc.Book_ID
AND bc.Copy_ID = t.Copy_ID
GROUP BY b.Title
ORDER BY Borrow_Count DESC;
```

Scenario 17: Find Available Books Only

Description:
This operation retrieves all books that currently have at least one available copy in the library inventory.

SQL Statement:

```
SELECT DISTINCT b.Book_ID, b.Title
FROM Books b, Book_Copies bc
WHERE b.Book_ID = bc.Book_ID
 AND bc.Status = 'Available';
```

Scenario 18: Reader Borrow Limit Check (Max 5 books)

Description:
This operation verifies that a reader does not exceed the maximum allowed number of active borrowings (5 books), as defined in the system requirements.

SQL Statement:

```
SELECT COUNT(*) AS Active_Borrows
FROM Transactions
WHERE (Member_ID = &member_id) AND (Status = 'Active');
```

Scenario 19: Delete Damaged Books (ADMIN)

Description:
This administrative operation removes damaged book copies that are no longer part of the active inventory.

SQL Statement:

```
DELETE FROM Book_Copies
WHERE Status = 'Damaged';
```

Scenario 20: Auto-Expire Reservations

Description:
This operation automatically updates the status of reservations that have passed their expiry date, ensuring that outdated reservations are no longer considered active in the system.

SQL Statement:

```
UPDATE Reservations
SET Status = 'Expired'
WHERE Expiry_Date < SYSDATE
AND Status = 'Active';
```

Scenario 21: Find Readers Who Never Borrowed

Description:
This operation identifies readers who have never performed any borrowing transactions in the system.

SQL Statement:

```
SELECT r.Member_ID, r.Name
FROM Readers r
WHERE NOT EXISTS (
SELECT 1
FROM Transactions t
WHERE t.Member_ID = r.Member_ID
);
```

Scenario 22: Top Fine Payees (Heavy Users)

Description:
This analytical query identifies readers who have accumulated the highest total fines over time.

SQL Statement:

```
SELECT r.Name, SUM(f.Fine_Amount) AS Total_Fines
FROM Readers r, Transactions t, Fines f
WHERE r.Member_ID = t.Member_ID
AND t.Transaction_ID = f.Transaction_ID
GROUP BY r.Name
ORDER BY Total_Fines DESC;
```

4. Application UI Demonstration (Done By Chaima Zaghouani)

To demonstrate the functionality of the Library Management System beyond individual SQL queries, we developed a Streamlit-based application that integrates all database operations into a unified user interface.

Instead of presenting separate screenshots for each SQL scenario, we showcase key application workflows that internally execute the corresponding SQL statements (scenarios 1–22). This approach provides a more realistic and comprehensive view of how the system operates in practice.

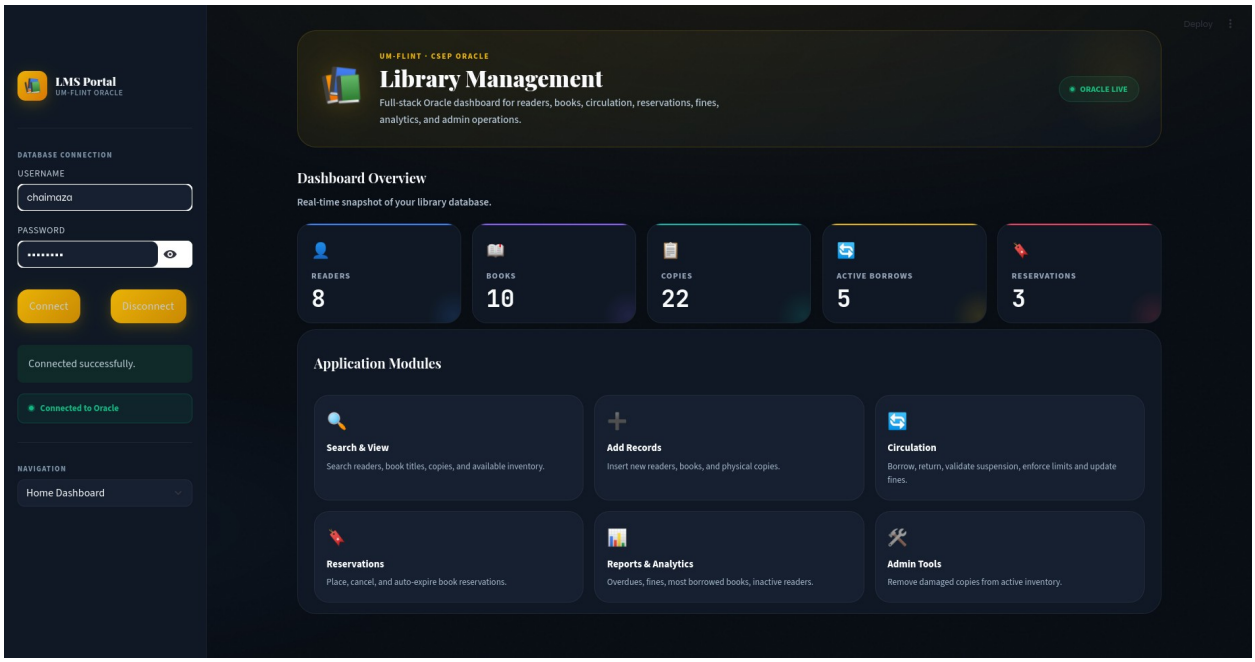
The application connects directly to the Oracle database and allows users to perform core operations such as:

- Viewing dashboard statistics (total readers, books, copies, active transactions, etc.)
- Registering new readers and adding books or copies
- Borrowing and returning books with automatic status updates
- Managing reservations and tracking overdue items
- Monitoring unpaid fines and generating analytical reports

The following screenshots illustrate key system interactions and confirm that all database operations were successfully implemented, tested, and integrated into an end-to-end workflow.

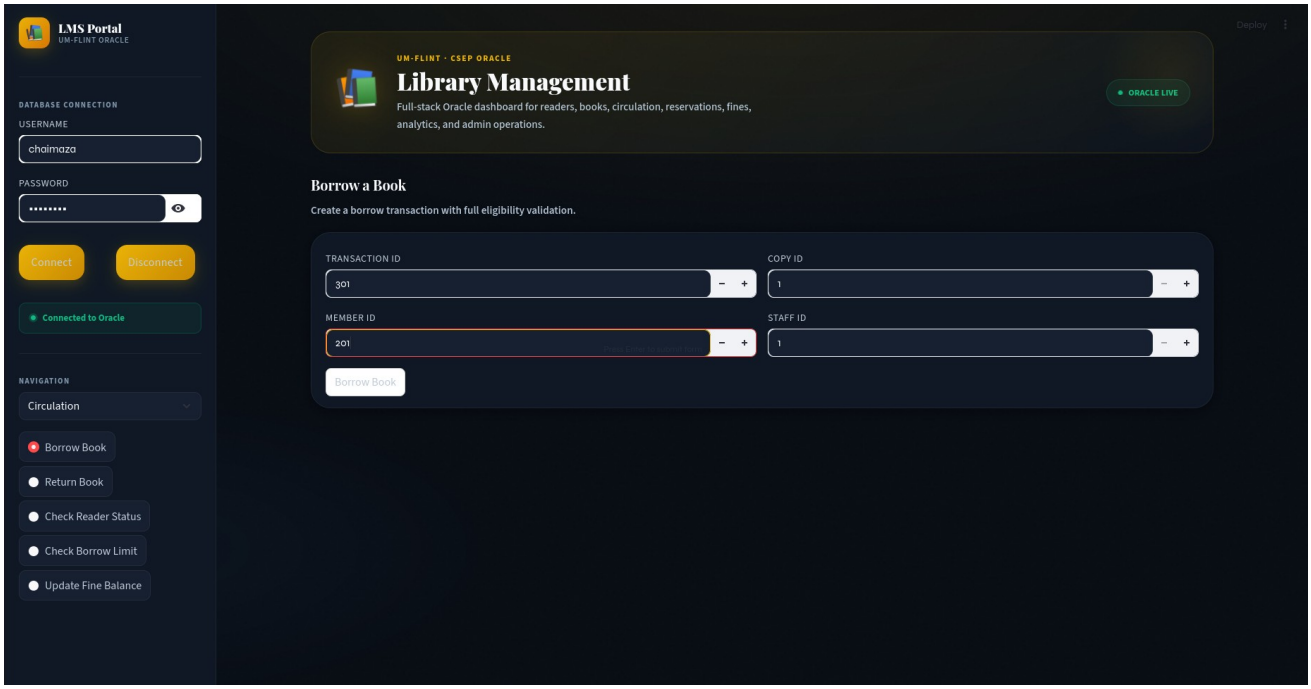
Dashboard Overview

The dashboard provides a real-time summary of the system, including total readers, books, copies, and active borrowing and reservation counts. This confirms correct execution of SQL aggregation queries (e.g., COUNT) and successful database connectivity.



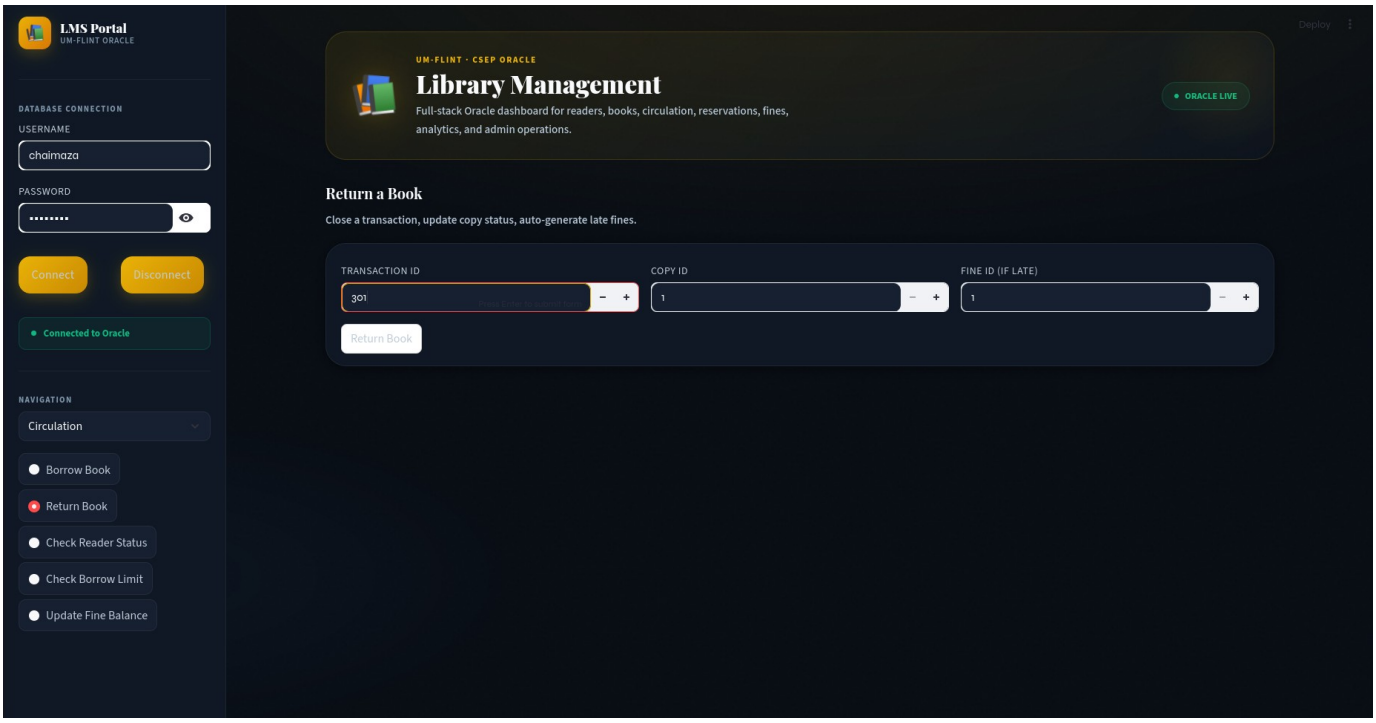
Borrow Book Operation

This operation demonstrates the borrowing workflow. When a book is borrowed, a new transaction is created and the corresponding book copy status is updated to "Borrowed". This confirms the correct implementation of INSERT and UPDATE operations, as well as enforcement of business rules.



Return Book Operation

This operation demonstrates the return workflow. The transaction is updated with a return date and status, and the book copy is marked as "Available". If applicable, fine logic is triggered. This confirms correct handling of updates and system state transitions.



Note:

- In addition to basic CRUD operations, we implemented advanced scenarios such as automatic fine generation using INSERT INTO SELECT, enforcement of business constraints (e.g., borrow limits, suspended users), and analytical queries (e.g., most borrowed books, top fine payers). These scenarios demonstrate the practical applicability of the LMS database in real-world environments.

- All operations were also validated directly in Oracle SQL Developer to ensure correctness and referential integrity.

- The Phase III scenarios were selected to balance routine library transactions with higher-level administrative and analytical operations. As a result, the LMS is demonstrated not only as a storage system, but also as a practical decision-support and workflow-management database.

- Additionally, the integration with the Streamlit application demonstrates the practical usability of the system in a real-world environment.

5. Group Member Contributions

For Phase III, both group members contributed to the design and implementation of the Library Management System operations, including SQL queries and application integration. Responsibilities were divided based on project scope and academic level to ensure efficient completion of all components. Each member participated in developing and testing SQL queries, while additional application-level implementation was carried out as part of the graduate-level requirements (implemented by Chaima Zaghouani). The final version was reviewed collaboratively to ensure correctness, consistency, and completeness of all scenarios and workflows.

Member	Contributions
Shayan Shoukat	Contributed to the design and implementation of core LMS SQL operations; participated in testing queries and validating outputs in Oracle SQL Developer; assisted in reviewing scenario logic and ensuring query correctness
Chaima Zaghouani	Designed and implemented the majority of SQL scenarios (retrieval, insertion, update, and analytical queries); developed and integrated the Streamlit application interface with the Oracle database; prepared and structured the Phase III report; tested and validated all system workflows and application functionality
Collaborative	Both members discussed and finalized the selection of scenarios to ensure comprehensive system coverage; jointly reviewed, debugged, and validated SQL queries; ensured consistency between the database design and implemented operations

Conclusion

This project demonstrates the complete design and implementation of a Library Management System (LMS) across all three phases, covering conceptual modeling, relational schema design, and practical database operations.

In Phase I, we developed a structured ER model that accurately represents real-world library entities and relationships, ensuring a clear and logical foundation for the system. In Phase II, we translated this design into a fully functional relational schema using Oracle SQL, implementing all tables, constraints, and sample data while maintaining referential integrity and consistency.

In Phase III, we implemented a comprehensive set of SQL operations that simulate real-world interactions, including data retrieval, insertion, updates, constraint enforcement, and analytical queries. Advanced functionalities such as automatic fine generation, borrow limit checks, and reservation management further demonstrate the robustness of the system.

The use of Oracle SQL features such as `SYSDATE` and `TO_DATE` enables dynamic handling of time-dependent operations, ensuring realistic and reliable system behavior. All queries were tested and validated in Oracle SQL Developer to confirm correctness and adherence to the defined schema.

In addition, the integration of the database with a Streamlit-based application provides a user-friendly interface that supports end-to-end workflows. This demonstrates how the LMS functions as a complete system, combining backend database logic with frontend interaction.

Overall, this project highlights the effectiveness of combining sound database design, comprehensive SQL implementation, and application-level integration to build a practical, scalable, and real-world data management solution.